

Take Attention as Gate: An Associative Recurrent Network-Based Intrusion Detection Method for Industrial Control Network

Ziyi Liu¹, Dengpan Ye¹, *Member, IEEE*, Changsong Yang², Yong Ding, Yueling Liu, Long Tang¹,
and Chuanxi Chen¹

Abstract—The Industrial Control Network (ICN), which is characterized by real-time responsiveness and reliability, plays a key role in increasing production speed, ensuring efficient processing, and managing industrial processes. Despite tremendous advantages, ICN inevitably struggles with some challenges, such as malicious user intrusion and hacker attacks. To detect malicious intrusions in ICN, Intrusion Detection Systems (IDS) have been deployed. However, network traffic in ICN often exhibits significant temporal periodicity, and computational resources are limited on edge nodes and infrastructure gateway devices. These characteristics pose significant challenges to the design and performance of IDS. To properly solve these problems, we design a new intrusion detection method for ICN. Specifically, we first design a novel neural network model called Associative Recurrent Network (ARN), which can properly handle the relationship between previous hidden state and current input. Then, we construct a novel intrusion detection method based on the ARN, which avoids gating conflicts in traditional Recurrent Neural Network (RNN), effectively captures the temporal characteristics of ICN traffic, and maintains slightly higher computational overhead than GRU, thus demonstrating good adaptability to industrial control networks. Subsequently, through theoretical analysis of computational complexity, we demonstrate that the proposed method achieves high computational efficiency, comparable to mainstream RNN methods and superior to Transformer methods. Finally, we implement a prototype system to evaluate detection accuracy. Experimental results show that our method achieves state-of-the-art performance on the industrial control systems datasets (ICS-ADD and SWaT) and the conventional network dataset (UNSW-NB15), with average accuracies of 98.93%, 95.57%, and 98.27%, respectively.

Index Terms—Network security, industrial control network, intrusion detection, associative recurrent network.

Received 28 October 2025; revised 28 January 2026 and 30 March 2026; accepted 24 April 2026. Date of current version 6 May 2026. This research was supported by New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0123603), the National Natural Science Foundation of China NSFC (No.62472325 and No.62562021) and the Hubei Provincial Key R&D Program, China (Grant No.2025BAB021). The associate editor coordinating the review of this article and approving it for publication was F. Paganelli. (*Corresponding author: Dengpan Ye.*)

Ziyi Liu, Dengpan Ye, Long Tang, and Chuanxi Chen are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: ziyi_liu@whu.edu.cn; yedp@whu.edu.cn; l_tang@whu.edu.cn; chencx@whu.edu.cn).

Changsong Yang, Yong Ding, and Yueling Liu are with the Computer Science and Information Security, Guilin University of Electronic Technology, Guilin 541004, China, and also with Guangxi Engineering Research Center of Industrial Internet Security and Blockchain, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: csyang@guet.edu.cn; stone_dingy@guet.edu.cn; ylliu@guet.edu.cn).

Digital Object Identifier 10.1109/TNSM.2026.3688982

I. INTRODUCTION

WITH the rapid development and integration of industrial informatization and automation, the Industrial Control System (ICS) has been widely applied, connecting numerous industrial infrastructures and devices through Industrial Control Network (ICN) [1]. Generally, ICN is characterized by real-time responsiveness and reliability, which plays a key role in production acceleration and production process management. Recently, due to the widespread adoption of 5G wireless networks [2], [3] and artificial intelligence, ICN has been widely used in natural gas pipeline systems, power transmission systems, industrial power generation systems, etc. As a result, both governments and enterprises have attached growing importance to the security and stability of ICN.

Despite tremendous advantages, ICN inevitably struggles with some security and technical challenges, such as network security protection. In the ICN, there are a large number of distributed industrial infrastructures and devices [4]. However, the security of ICN physical isolation is broken due to the fusion and development of information technology [5], thus bringing a series of new security problems. Meanwhile, ICN might be frequently invaded by malicious users and hackers, which not only causes damage to ICN but also poses a threat to the entire ICS. Therefore, the security protection of ICN is particularly important. However, most of the traditional network security defense methods, such as firewalls and vulnerability scanning, can only achieve passive defense.

To achieve active defense in the ICN, intrusion detection [6] technologies have been widely applied. Intrusion detection is a typical active defense technology that aims to detect network intrusions by analyzing network traffic data, such as user behavior data, behavior logs and audit data. By deploying intrusion detection systems, network security managers can monitor the entire ICN without affecting the network performance, thereby providing ICN devices and systems with real-time security protection. Therefore, intrusion detection attracts lots of attention, and has produced a large number of theoretical and engineering results. Roughly, the existing intrusion detection methods can be divided into three categories: statistics-analysis-based intrusion detection, time-series-based intrusion detection and deep-learning-based intrusion detection.

Although plenty of intrusion detection methods have been proposed, there are still some internal deficiencies when they are directly applied to the ICN. Unlike traditional enterprise

network traffic, which typically exhibits diverse characteristics and a wide variety of protocols [7], the traffic generated by ICN exhibits significant differences in communication structure, protocol characteristics, and security constraints. Specifically, ICN communications are typically based on industrial protocols such as Modbus, DNP3, and IEC-104, and are characterized by strong periodicity and clear protocol semantics [8]. Moreover, ICNs often operate in critical infrastructure such as electrical networks, transportation, and petrochemicals, placing higher real-time requirements and stricter computing resource constraints on intrusion detection security mechanisms [9]. These differences make it difficult for traditional intrusion detection methods to be directly applied to the actual ICN environment. In particular, the strong periodicity of ICN traffic places higher demands on the temporal learning capabilities of the detection model, requiring the model to effectively capture and characterize periodic behavior. However, existing intrusion detection methods still face the following limitations: On the one hand, traditional statistics-analysis-based intrusion detection methods require substantial computational overhead when handling large-scale data, while time-series-based intrusion detection methods (excluding deep learning methods) mainly focus on temporal dependencies within the data and often overlook the integration of other relevant features for comprehensive analysis. On the other hand, deep learning-based recurrent neural networks [10] (such as Gated Recurrent Unit [11] and Long Short-Term Memory [12]) have demonstrated a certain ability to model temporal structures and have been applied in ICN scenarios. However, when integrating previous hidden state with current input, these models often suffer from information competition between gating mechanisms—commonly referred to as gating conflicts—which can degrade overall performance [13], [14]. Although Transformer models exhibit superior temporal modeling capabilities and can capture the periodic patterns inherent in ICN traffic, their complex architecture and high computational overhead have, to some extent, hindered their widespread adoption on resource-constrained edge nodes and gateway devices within ICN environments. Therefore, designing an intrusion detection approach that balances effective temporal learning capabilities with low computational overhead has become a critical challenge in the security protection of ICN.

To address the aforementioned challenges, we propose a novel network structure called the Associative Recurrent Network (ARN). ARN is designed to achieve efficient representation learning of ICN traffic with computational resource overhead comparable to that of the Gated Recurrent Unit (GRU), thereby enhancing intrusion detection performance in ICN. Considering that ICN traffic often exhibits pronounced temporal periodicity, there is usually a strong correlation between the previous hidden state and the current input, where the former provides essential cues for determining whether the current data instance is anomalous. In our approach, ARN employs a Single Attention (S-ATT) mechanism to learn the correlation between previous hidden state and current input, avoiding the gating conflicts that may occur in traditional recurrent neural networks. Meanwhile, ARN has low model complexity, and its computational overhead is only

slightly higher than that of GRU, thereby balancing gating-conflict mitigation with computational efficiency. Benefiting from these two design aspects, ARN can effectively detect traffic in resource-constrained ICN environments, providing a new solution that combines performance and practicality for ICN intrusion detection. It also offers new insights for subsequent ICN intrusion detection research.

A. Contributions

In this paper, we investigate a fundamental but challenging issue, i.e., how to improve the performance of intrusion detection in the ICN. Specifically, we design a novel RNN model, namely, Associative Recurrent Network (ARN). Then, we adopt ARN to propose a new intrusion detection method for ICN. Therefore, the main contributions of this paper can be summarized as the following three folds.

- We design a new recurrent neural network called ARN for intrusion detection in industrial control networks, which does not rely on gating units for effective temporal learning. Compared with other types of RNN models, ARN eliminates the conflict risk between gated units and can better characterize the long-term and short-term dependencies of data.
- We design a Single Attention (S-ATT) mechanism to retain important historical information in the ARN. S-ATT uses the attention mechanism to directly learn the correlation between the previous hidden state and the current input, allowing ARN to abandon the traditional gated unit structure and more reasonably preserve previous hidden state.
- We compare the ARN method with mainstream RNN and Transformer methods in terms of theoretical computational complexity. The results show that ARN achieves the same time complexity as mainstream RNN methods while outperforming Transformers. Furthermore, we develop a prototype system and conduct simulations to evaluate its performance. Experimental results demonstrate that our method achieves accuracy of 98.93%, 95.57%, and 98.27% on the ICS-ADD, SWaT, and UNSW-NB15 datasets, respectively.

B. Organization

The rest of this paper is organized as follows. In Section III, we provide a detailed introduction to GRU and Single Attention (S-ATT). Subsequently, in Section IV and Section V, we present the structure of ARN and design an ARN-based intrusion detection method, respectively. Then, in Section VI, we provide a theoretical computational complexity comparison of the ARN-based intrusion detection method and GRU-based intrusion detection method. Next, we develop a prototype implementation for our proposed method and present the performance evaluation in Section VII. Finally, we provide a brief conclusion in Section VIII.

II. RELATED WORK

Generally, existing intrusion detection methods can be roughly summarized into three categories: statistics-analysis-based intrusion detection method [15], time-series-based

intrusion detection method [16] and deep-learning-based intrusion detection method [17].

A. Statistics-Analysis-Based Intrusion Detection Method

Wu et al. [18] proposed an information entropy Intrusion Detection System (IDS) based on improved sliding window strategy. By improving the optimizing conditions of the decision and sliding window strategy, their method could enhance the IDS accuracy. To detect invisible attacks in ICS, Hu et al. [19] represented the ordered content in the residual by Permutation Entropy (PE). According to certain rules presented by sensor measurement values Zhou et al. [20] proposed an industrial information system attack detection scheme based on permutation entropy. Liao et al. [21] designed a gray-entropy-based IDS, which could analyze and cluster the traffic gray level. Meanwhile, their designed system had higher intrusion detection accuracy for both known and unknown traffic attacks. However, the aforementioned intrusion detection methods are limited by entropy and relative uncertainty measures, resulting in exponential time complexity [22]. When there is a lot of noise in industrial network traffic and the traffic scale is large, they are difficult to detect attack traffic.

Liang et al. [23] divided the data weighted distance and safety factor of each node in the network according to the priority threshold of data attribute characteristics, and designed an ICN intrusion detection method based on multi-feature data clustering optimization model. Zhang et al. [25] combined traffic Similarity metrics with Density-Based Spatial Clustering of Applications with Noise (SDBSCAN) methods to construct a new network intrusion detection framework. Mittal et al. [24] implemented a new clustering method using a new variant of the gravitational search algorithm and used it for detection in the industrial Internet of Things (IIoT). However, clustering algorithms usually require a lot of computational overhead and memory usage overhead when facing large-scale data. Therefore, when facing large-scale industrial network data, clustering-algorithm-based intrusion detection methods are difficult to meet the defense needs of the industry.

In summary, we can see that statistics-analysis-based intrusion detection methods require many mathematical calculations and analyses, resulting in a heavy computational burden. Meanwhile, while processing high-dimensional and irregularity data, the accuracy of the above methods is insufficient.

B. Time-Series-Based Intrusion Detection Method

Bozdal et al. [26] presented a wavelet-based ingress method for the Controller Area Network (CAN). By analyzing changes in network traffic data, their proposed method could precisely determine attack patterns. To describe the intrusion attacks, Miao et al. [27] studied attack signals in ICS intrusions. Through frequency-domain analysis of network traffic, they proposed linear and nonlinear attack signal estimators (LASE and NASE) and analyzed the impact of frequency-domain attack signals on LASE performance. Fouladi et al. [28] adopted frequency-based discrete wavelet transform and discrete Fourier transform to design a naive Bayesian classifier, which could exactly distinguish attack traffic data from normal traffic data. However, the above wavelet-based detection

methods are limited by noise interference and the selection of characteristic signals, resulting in detection deviation.

Ye et al. [29] adopted Hurst parameter estimation to design a Fractional Fourier Transform (FRFT)-based intrusion detection method. Meanwhile, they developed a prototype implementation to test the performance. Experimental results showed that their proposed method was not affected by non-stationary time series in Hurst estimation. Huang et al. [30] simultaneously extracted time-domain and frequency-domain features from the sensor measurement data required for closed-loop control of ICS, and used these features to establish a hidden Markov model to implement intrusion detection. Zhang et al. [31] developed a method for predicting dynamic power system intent that combined attribute selection and clustering with a Dynamic Hidden Markov Model (DHMM) to capture the evolving nature of cyberattack behavior. However, the Fourier-transform-based detection methods are susceptible to boundary effects and the hidden Markov transform is limited by the dependence on the previous. Therefore, it is difficult to achieve accurate industrial network intrusion detection.

In addition, time-series-based (non-deep learning) intrusion detection methods typically focus on temporal dependencies in data, while rarely incorporating other data features for comprehensive analysis. This limitation significantly restricts their applicability in intrusion detection.

C. Deep-Learning-Based Intrusion Detection Method

Li et al. [32] used a multi-Convolutional Neural Network (multi-CNN) fusion method to perform intrusion detection on industrial IIoT traffic. Abdelaty et al. [33] constructed a deep learning anomaly detection in ICS using a 2-branch Deep Neural Network (DNN). Their method learned the changes in the behavior of ICS through a few data samples and gradient updates, thus effectively improving the accuracy and robustness of intrusion detection. Yang et al. [34] combined Levenberg-Marquardt (LM) and Back Propagation Neural Network (BPNN) to present a new IDS. In their proposed method, LM was utilized to optimize the weights of BPNN, thus improving accuracy. Sood et al. [35] proposed a two-stage IDS, which could achieve data dimensionality reduction, and was compatible with the ETSI-NFV standard 5G architecture. Experimental results showed that their method has higher accuracy in traffic detection. However, these deep learning methods fail to leverage the temporal sequences of network traffic, limiting their ability to detect time-sensitive attacks.

Jadidi et al. [36] proposed a two-stage causal anomaly detection method for ICS. It first performed attack detection using an LSTM-based classifier, and then analyzed causal relationships among ICS logs to diagnose the potential future impact of attacks. Xu et al. [37] utilized GRU, Multi-layer Perceptron (MLP) and softmax layer to design a new IDS. To address the curse of dimensionality and the trade-off between low false alarm rate and high detection rate, Mushtaq et al. [38] proposed a two-stage detection model, AE-LSTM, which first performed feature dimensionality reduction using an AutoEncoder (AE) and then conducted classification

TABLE I
COMPARISON OF RELATED RESEARCH ON INTRUSION DETECTION METHODS BASED ON DEEP LEARNING
AND PARTIALLY NON-DEEP LEARNING

Type	Method	Target Network	Feature Reduction	Main Algorithms for Detection	Use of Time-Series Information	No Gate Conflict
Non-Deep Learning	Hu et al. [19]	ICS	None	PE	✓	✓
	Zhang et al. [25]	ICS	Grouping	Conditional Entropy, Grouping, DBSCAN	✓	✓
	Miao et al. [27]	ICS	None	LASE, NASE	✓	✓
	Zhang et al. [31]	ICS	Random Forest, Density Clustering	Random Forest, Density Clustering, DHMM	✓	✓
Deep Learning	Li et al. [32]	ICS	None	CNN	✓	✓
	Abdelaty et al. [33]	ICS	None	2-branch DNN	✓	✓
	Yang et al. [34]	IoT	None	LM, BPNN	✓	✓
	KSood et al. [35]	IoT	AE	AE, BPNN	✓	✓
	Jadidi et al. [36]	ICS	None	LSTM	✓	✓
	Xu et al. [37]	General	None	GRU	✓	✓
	Mushtaq et al. [38]	General	AE	AE, LSTM	✓	✓
	Zhang et al. [39]	ICS	None	Fast Fourier Transform, Bidirectional GRU, CNN, GAT	✓	✓
	Ullah et al. [40]	IoT	None	Transformer	✓	✓
	Ours	ICS	None	ARN	✓	✓

TABLE II
LIST OF IMPORTANT SYMBOL DESCRIPTIONS. THE SYMBOLS IN THE TABLE ARE LISTED IN THE ORDER THEY APPEAR IN THE EQUATIONS:
FIRST FROM TOP TO BOTTOM IN THE LEFT COLUMN, THEN FROM TOP TO BOTTOM IN THE RIGHT COLUMN

Symbol	Description	Symbol	Description
x_t	Current input	a_h / a_x	The weight obtained by calculating the similarity between q^1 and each input k
R_t	Reset gate	d	The dimensions of q , k and v
Z_t	Update gate	$\hat{a}_h / \hat{a}_x$	a_h / a_x after softmax activation
W_R	Weights for reset	h_{t-1}^u	Supplementary matrix after S-ATT calculation
W_Z	Weights for update	$W^x / W^{h_{t-1}}$	The weight matrices of x_t / h_{t-1}
σ	Sigmoid activation function	h_{cat}	Concatenate h_{t-1} and x_t into a format suitable for S-ATT input
$\tilde{h}_t$	Candidate set	cat	Splicing method in Python
h_{t-1}	Previous hidden state	D_{in}	Input dimensions of ARN, GRU, and LSTM
h_t	Current hidden state	H	Hidden layer dimension of ARN, GRU, and LSTM
q^1 / q^2	The query of h_{t-1} and X_t	T	Input sequence length
k^1 / k^2	The key of h_{t-1} and X_t	D	Embedding dimension of Transformer
v^1 / v^2	The value of h_{t-1} and X_t	C_{out}	Convolution output channels of Transformer
$W^q / W^k / W^v$	The weight matrices of $q / k / v$	K	Convolution kernel size of Transformer

through an LSTM. Experimental results demonstrated that AE-LSTM outperforms AE-BiLSTM and other related methods. Zhang et al. [39] combined multi-scale periodicity extraction with spatiotemporal Graph Attention Networks (GAT) to address the difficulty of traditional methods in capturing spatiotemporal correlations and periodicity in system data. Ullah [40] developed a Transformer Neural Network (TNN)-based intrusion detection system tailored to mitigate the detection limitations caused by imbalanced data. However, most of the RNN-based intrusion detection methods have mutual influence between gating units, which makes it difficult to effectively manage the potential connections between long-term and short-term dependencies of data, limiting the accuracy of intrusion detection. Although Transformer-based intrusion detection methods can avoid the problem of gating conflicts, their high memory and time overhead during operation significantly increase the deployment cost in ICS environments, thereby limiting their practical applicability.

In summary, although deep learning-based intrusion detection methods can effectively represent large-scale network traffic data, there is still an urgent need for a new approach that can avoid gating conflicts, fully exploit the temporal dependencies of network traffic for representation learning, and maintain memory and computational costs comparable to traditional recurrent neural networks (such as GRU), in order to further enhance intrusion detection performance in ICS environments. To this end, we will conduct in-depth research on this issue and propose a new deep-learning-based intrusion detection method. We select two representative methods from statistics-analysis-based intrusion detection methods, two from time-series-based intrusion detection methods, and together with deep-learning-based intrusion

detection methods, constructed Table I to compare the different methods.

III. PRELIMINARIES

In this section, we introduce some preliminaries for developing the new intrusion detection method, including GRU and our newly designed S-ATT. The GRU is a type of recurrent neural network that efficiently captures long-term dependencies in sequences through gating mechanisms. S-ATT is a single attention mechanism inspired by self-attention, which learns the relationship between the previous hidden state and the current input to construct a supplementary matrix, thereby enhancing the models ability to retain key information. Moreover, We summarize the important symbols used in this paper in Table II.

A. Gated Recurrent Unit

Gated Recurrent Unit (GRU) is an improved model of RNN that can effectively transmit long-term dependency information. In contrast to Long Short-Term Memory (LSTM), which is equipped with three gate structures, GRU only contains two gates: the reset gate and the update gate, as illustrated in Figure 1. As a result, GRU requires fewer parameters and can reduce the risk of overfitting during training. In GRU, the reset gate is utilized to decide the proportion of hidden state that flows into the current candidate set at the previous. Meanwhile, the update gate has two functions. On the one hand, it is utilized to control the proportion of the content that should be discarded at the previous. On the other hand, it can also determine the proportion of the content that needs to be retained in the current candidate set.

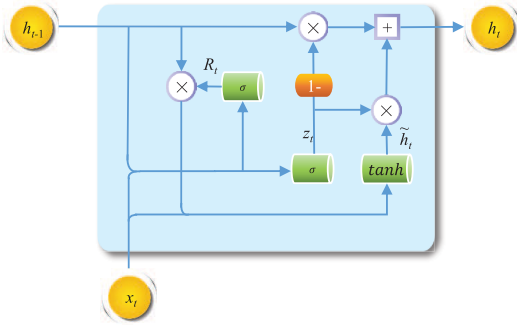


Fig. 1. The structure of the GRU, where the update gate and reset gate jointly regulate the preservation and update of hidden states, enabling the model to capture long-term dependencies in sequential data.

As shown in Figure 1, the related formulas for calculating GRU are as follows.

Firstly, R_t and Z_t must be initialized according to Formula (1) and Formula (2) as follows:

$$R_t = \sigma([h_{t-1}, x_t] \cdot W_R) \quad (1)$$

$$Z_t = \sigma([h_{t-1}, x_t] \cdot W_Z) \quad (2)$$

Secondly, $\tilde{h}_t$ of GRU is controlled by R_t . Meanwhile, $\tilde{h}_t$ can be described by Formula (3) as follows:

$$\tilde{h}_t = \tanh(W_{\tilde{h}} \cdot [x_t, R_t \times h_{t-1}]) \quad (3)$$

Finally, GRU updates h_t according to Formula (4):

$$h_t = Z_t \times \tilde{h}_t + (1 - Z_t) \times h_{t-1} \quad (4)$$

From the above formulas and Figure 1, we find that the Z_t and the R_t conflict in the retention proportion of the hidden state in the previous. Specifically, the reset of the current hidden state in GRU is provided by the R_t and Z_t . The role of the R_t is to determine the proportion of the previous hidden state retained in the current candidate set. However, the Z_t can ignore part of the previous hidden state that the R_t has determined to retain. Additionally, the Z_t arbitrarily retains the previous hidden state in the process of resetting the current hidden state. Therefore, there is a certain conflict between the Z_t and the R_t , making it difficult to effectively manage the retention relationship between the previous hidden state and the current input.

B. Single Attention

To effectively solve the problem of how to determine the retention relation between the hidden state of the previous and the input of the current, we use the idea of Self-attention [41] to propose a one-way self-attention mechanism solution called Single Attention (S-ATT). S-ATT can effectively manage the retention relationship between the h_{t-1} and the x_t . The implementation of the S-ATT module is illustrated in Figure 2.

First, we initialize q^n, k^n, v^n ($n = 2$) to facilitate subsequent calculations.

$$q^1 = W^q h_{t-1}, q^2 = W^q x_t \quad (5)$$

$$k^1 = W^k h_{t-1}, k^2 = W^k x_t \quad (6)$$

$$v^1 = W^v h_{t-1}, v^2 = W^v x_t \quad (7)$$

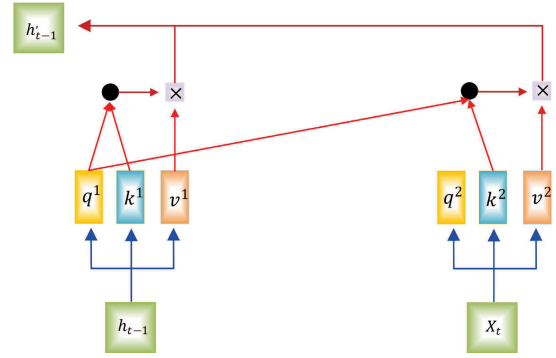


Fig. 2. The structure of the S-ATT mechanism, which learns the dependency between the previous hidden state h_{t-1} and the current input x_t to generate a supplementary matrix h'_{t-1} .

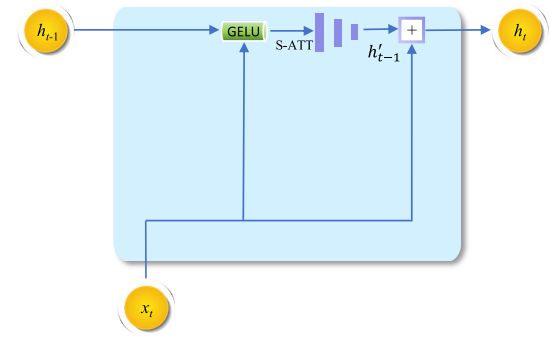


Fig. 3. The structure of ARN, which utilizes S-ATT to learn dependencies between the previous hidden state and the current input, thereby effectively capturing the temporal characteristics of the data.

Then, we calculate the correlation between h_{t-1} and x_t , and get the corresponding weight matrices a_h and a_x .

$$a_h = \frac{q^1 \cdot k^1}{\sqrt{d}}, a_x = \frac{q^2 \cdot k^2}{\sqrt{d}} \quad (8)$$

$$\hat{a}_h, \hat{a}_x = \text{softmax}(a_h, a_x) \quad (9)$$

Finally, we calculate and fuse it into a new supplementary matrix h'_{t-1} based on the proportion of weights.

$$h'_{t-1} = \hat{a}_h v^1 + \hat{a}_x v^2 \quad (10)$$

IV. ASSOCIATIVE RECURRENT NETWORK

To address the problems of GRU that described in Section III-A, we propose a new RNN method called ARN. ARN utilizes the S-ATT module to unidirectionally learn the relationship between the previous hidden state and the current input, thereby constructing a supplementary matrix to supplement the current input and reset the current hidden state. The reset current hidden state contains important historical information unknown to the current input and highlights the important parts of information in the current input. Therefore, ARN can better manage the information retention relationship between the previous hidden state and the current input, thereby avoiding the gate conflict problem in GRU and enabling more efficient intrusion detection for ICN traffic. The overall structure of ARN is illustrated in Figure 3.

Firstly, we require to initialize x_t and h_{t-1} with W^x and $W^{h_{t-1}}$ while making x_t and h_{t-1} have the same matrix dimensions, which is convenient for subsequent correlation calculation. Then, x_t and h_{t-1} can be initialized as Formula (11) and Formula (12), respectively.

$$x_t = W^x x_t \quad (11)$$

$$h_{t-1} = W^{h_{t-1}} h_{t-1} \quad (12)$$

Secondly, we use cats to concatenate h_{t-1} and x_t together, and construct the S-ATT input through the activation function GELU, as shown in Formula (13).

$$h_{cat} = GELU(cat(h_{t-1}, x_t)) \quad (13)$$

Subsequently, we utilize S-ATT to learn the relationship between h_{t-1} and x_t , thus constructing a new supplementary matrix h'_{t-1} . The matrix h'_{t-1} contains part of the important content in the current input (i.e., the portion of x_t that is related to h_{t-1}), and the information in the previous hidden state that is unknown to the current input, as shown in Formula (14).

$$h'_{t-1} = S-ATT(h_{cat}) \quad (14)$$

Finally, we supplement h'_{t-1} to x_t to achieve the reset of h_t . The reset h_t contains previous hidden state that is unknown in the input x_t at the current time and highlight the important content in the current input, so as to better manage the retention relationship between previous hidden state and current input. Then, h_t is computed as in (15).

$$h_t = h'_{t-1} + x_t \quad (15)$$

In summary, ARN can effectively learn the highlighted important parts of the current content in feature learning by constructing a supplementary matrix and fully consider the impact of the unknown previous hidden state on the current.

V. ARN-BASED INTRUSION DETECTION METHOD

In this section, we utilize ARN to build a new intrusion detection method for ICN.

A. System Model

In this paper, we focuses on the development of intrusion detection methods for Industrial Control Networks (ICNs). With the rapid expansion of ICNs, their applications and adoption continue to grow. Notably, ICN network traffic is characterized by large scale, pronounced periodicity, and well-defined protocol semantics. Moreover, computational resources on edge nodes and gateway devices remain limited. These characteristics present significant challenges to the generalizability and accuracy of intrusion detection methods.

To address the above problems, we propose a new intrusion detection method, as illustrated in Figure 4. Specifically, a novel RNN model called ARN is proposed. Compared with other DNN models, ARN can not only learn and characterize network traffic data through time series but also better handle the retention relationship between previous hidden state data and current input. Subsequently, we utilize ARN to build a new intrusion detection method for ICN. By taking the advantage of ARN, our novel proposed method can greatly enhance the intrusion detection accuracy.

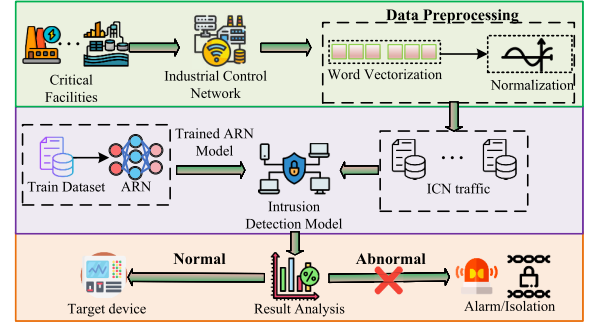


Fig. 4. The system model. In the ICN, the collected traffic is first preprocessed and then fed into the trained ARN intrusion detection model for detection. If the traffic is classified as normal, it is allowed to continue flowing within the system; if deemed anomalous, an alert is triggered.

B. The Proposed Method

In this section, we introduce the implementation of ARN-based intrusion detection method for ICN in detail.

Algorithm 1 ARN-Based Intrusion Detection Method

Require: Raw ICS dataset $\mathcal{D}$, real-time ICS data X_{real} , number of runs n

Ensure: Detection result S

Model Training and Evaluation

- 1: $\mathcal{D}' \leftarrow \text{PREPROCESS}(\mathcal{D})$
- 2: $(X_{\text{train}}, X_{\text{val}}, X_{\text{test}}) \leftarrow \text{SPLIT}(\mathcal{D}')$
- 3: **for** $i = 1$ to n **do**
- 4: $\text{ARN}_i \leftarrow \text{TRAIN}(X_{\text{train}})$
- 5: $R_i \leftarrow \text{EVALUATE}(\text{ARN}_i, X_{\text{val}})$
- 6: **end for**
- 7: $\text{ARN} \leftarrow \text{SELECTBEST}(\{\text{ARN}_i\}_{i=1}^n)$
- 8: $R \leftarrow \text{EVALUATE}(\text{ARN}, X_{\text{test}})$

Intrusion Detection

- 9: $X'_{\text{real}} \leftarrow \text{PREPROCESS}(X_{\text{real}})$
- 10: $S \leftarrow \text{ARN}(X'_{\text{real}})$
- 11: **if** S is normal **then**
- 12: **return** pass
- 13: **else**
- 14: **return** attack
- 15: **end if**

To describe the proposed method more clearly, we first define the main symbols and operators used in Algorithm 1. Specifically, the training, validation, test, and real-time traffic datasets are denoted by X_{train} , X_{val} , X_{test} , and X_{real} , respectively. The intrusion detection result is denoted by S , the raw ICS dataset is $\mathcal{D}$, and the number of runs is denoted by n . In addition, $\text{PREPROCESS}(\cdot)$ denotes the data preprocessing operation, $\text{SPLIT}(\cdot)$ denotes the dataset partition operation, $\text{TRAIN}(\cdot)$ denotes the model training operation, $\text{EVALUATE}(\cdot)$ denotes the performance evaluation operation, and $\text{SELECTBEST}(\cdot)$ denotes the model selection operation. The corresponding evaluation result is denoted by R . Based on these symbols and operators, the proposed intrusion detection method is described in Algorithm 1.

TABLE III
COMPARISON OF FLOPS AND TIME COMPLEXITY OF ARN,
GRU, LSTM AND TRANSFORMER

Method	FLOPs	Time complexity
ARN	$T \cdot (2D_{in}H + 20H^2)$	$O(T \cdot (D_{in}H + H^2))$
GRU	$T \cdot (6D_{in}H + 6H^2)$	$O(T \cdot (D_{in}H + H^2))$
LSTM	$T \cdot (8D_{in}H + 8H^2)$	$O(T \cdot (D_{in}H + H^2))$
Transformer	$8D^2 + 4T^2D + 4KTDC_{out}$	$O(T^2D + KTDC_{out} + D^2)$

As shown in Algorithm 1, the proposed ARN-based intrusion detection method consists of two stages: model training and evaluation, and intrusion detection.

Model training and evaluation, the ICS dataset used for model training and evaluation is first collected and preprocessed, including feature vectorization and data normalization. Here, feature vectorization refers to converting categorical fields in network traffic data, such as protocol type, function code, and status flag, into fixed-dimensional numerical vectors. The processed data are then divided into the training set X_{train} , validation set X_{val} , and test set X_{test} . After that, the training set is used to train the intrusion detection model, the validation set is used to assist model selection and hyperparameter tuning, and the test set is used to evaluate the final detection performance of the trained model. The ARN proposed in this paper serves as the core component of the intrusion detection model and is designed to effectively perform intrusion detection on ICN traffic. Specifically, when the current traffic input x_t is fed into ARN, the relationship between the previous hidden state h_{t-1} and the current input x_t is learned through the S-ATT mechanism. In this way, the model can extract the important information in the current input x_t that is closely related to the previous hidden state h_{t-1} , and combine it with the key information in the previous hidden state h_{t-1} that is unknown to the current input x_t , thereby forming the supplementary matrix h'_{t-1} . By adding the supplementary matrix h'_{t-1} to the current input x_t , ARN forms the current hidden state h_t , which both highlights the important information in the current input and incorporates historical information from the previous hidden state that is unknown to the current input.

Intrusion detection, real-time ICS traffic is collected and preprocessed in the same manner as in the model training and evaluation stage, and then fed into the trained ARN model for analysis. If the detection result is normal, the communication is allowed to continue; otherwise, the malicious traffic is isolated and an alarm is triggered.

VI. COMPUTATIONAL COMPLEXITY ANALYSIS

In this section, we compare the theoretical computational complexity of the proposed ARN method with those of GRU, LSTM, and Transformer, as summarized in Table X. It should be noted that the computational complexity of the Transformer in this study is calculated based on the Transformer used in the TNN [40] model, while that of the LSTM is calculated based on the LSTM used in the ICS-CAD [36] model. The formulas for computing ARN and GRU can be found in Sections III-A and Section IV, respectively, whereas the computation formulas for LSTM [36] and Transformer [40] can be referred to in the corresponding literature implementations.

For ARN, we assume that the input sequence has a length of T , the input dimension is D_{in} , and the hidden layer dimension is H . At each time step, the computational cost mainly comes from the linear transformations and the S-ATT module. The FLOPs for linear mappings are $2D_{in}H$ and $2H^2$, respectively. Assuming the S-ATT operates on the concatenated vector of h and x with sequence length 2, the S-ATT module contributes approximately $18H^2$ FLOPs. Therefore, the per-step computational cost is $2D_{in}H + 20H^2$. For the entire sequence, the total recurrent computation is $T \cdot (2D_{in}H + 20H^2)$. It can be seen that the time complexity of ARN is $O(T \cdot (D_{in}H + H^2))$.

For GRU, we assume that the input sequence has a length of T , the input dimension is D_{in} , and the hidden layer dimension is H . The primary computational cost at a single time step arises from matrix multiplication operations, including the computations for the update gate, reset gate, and candidate hidden state. According to the formula for the floating-point operations of matrix multiplication, the computational cost at a single time step can be expressed as $6D_{in}H + 6H^2$. For the forward pass over the entire sequence, the total number of FLOPs for GRU is $T \cdot (6D_{in}H + 6H^2)$. It can be seen that the time complexity of GRU is $O(T \cdot (D_{in}H + H^2))$.

For LSTM, we assume that the input sequence has a length of T , the input dimension is D_{in} , and the hidden layer dimension is H . The main computation at each time step comes from the linear mapping of the four gates (forget gate, input gate, candidate memory gate, and output gate). The FLOPs for each gate are $2D_{in}H + 2H^2$, and the total FLOPs for all four gates are $8D_{in}H + 8H^2$. For the entire sequence, the total FLOPs are $T \cdot (8D_{in}H + 8H^2)$. It can be seen that the time complexity of LSTM is $O(T \cdot (D_{in}H + H^2))$.

For Transformer Encoder with embedding dimension D , sequence length T , convolution output channels C_{out} , and convolution kernel size K , the computational cost mainly comes from three parts: the multi-head attention module, the convolutional sublayer, and the final fully connected layer. Since the ARN, GRU, and LSTM modules only consider the computational cost of the core model itself and do not include the task-specific fully connected layers used for classification, this fully connected part is also excluded from the calculation. The FLOPs of the multi-head attention module are $8D^2 + 4T^2D$. The FLOPs of the convolutional sublayer are $4KTDC_{out}$. Therefore, the total FLOPs of the Transformer core module are $8D^2 + 4T^2D + 4KTDC_{out}$. It can be seen that the time complexity of Transformer is $O(T^2D + KTDC_{out} + D^2)$.

From the perspective of theoretical computational complexity, the main computation at each time step of LSTM, GRU, and ARN comes from the linear mappings between the input and the hidden state, with a computational cost of $O(D_{in}H + H^2)$ per step. For the entire sequence, the total computational complexity is $O(T \cdot (D_{in}H + H^2))$, which grows linearly with the sequence length. In contrast, Transformer, in addition to linear projections $O(D^2)$, requires multi-head self-attention computation, which involves pairwise correlations between sequence elements and contributes an additional $O(T^2D)$. Moreover, the convolutional sublayer further adds $O(KTDC_{out})$. When the sequence length T is large, the T^2D term from attention computation quickly dominates over the

TABLE IV

FEATURES USED IN THE SWaT DATASET AND UNSW-NB15 DATASET IN THE EXPERIMENT. THE MEANING OF THESE FEATURES CAN BE FOUND IN THE LITERATURE CITED IN THE SWaT DATASET [43] AND THE UNSW-NB15 DATASET [44]

Dataset	Network Features	Dataset	Network Features
SWaT	Timestamp, FIT101, LIT101, MV101, P101, P102, AIT201, AIT202, AIT203, FIT201, MV201, P201, P202, P203, P204, P205, P206, DPIT301, FIT301, LIT301, MV301, MV302, MV303, MV304, P301, P302, AIT401, AIT402, FIT401, LIT401, P401, P402, P403, P404, UV401, AIT501, AIT502, AIT503, AIT504, FIT501, FIT502, FIT503, FIT504, P501, P502, PIT501, PIT502, PIT503, FIT601, P601, P602, P603	UNSW-NB15	Sport, Dsport, Proto, State, Dur, Sbytes, Dbytes, Sttl, Dttl, Sloss, Dloss, Service, Sload, Dload, Spkts, Dpkts, Swin, Dwin, Stepb, Dtepb, Smeansz, Dmeansz, Trans_depth, Res_body_len, Sjit, Djit, Sintpkt, Dintpkt, Tcprrt, Synack, Ackdat, Is_sm_ips_ports, Ct_state_ttl, Ct_flw_http_mthd, Is_ftp_login, Ct_ftp_cmd, Ct_srv_src, Ct_srv_dst, Ct_dst_ltm, Ct_src_ltm, Ct_src_dport_ltm, Ct_dst_sport_ltm, Ct_dst_src_ltm

TABLE V

FEATURES USED IN THE ICS-ADD DATASET AND THEIR DESCRIPTIONS

Network Features	Description
Sport	The source port number of the connection.
Dport	The destination port number used by the receiving host.
Proto	The transport or application-layer protocol used in the flow.
Dur	The total time duration of the flow.
Pkt_count	The total number of packets observed within the entire flow.
Spkts	The number of packets sent from the source host to the destination host.
Dpkts	The number of packets sent from the destination host back to the source host.
Sbytes	The total number of bytes transmitted from the source to the destination.
Dbytes	The total number of bytes transmitted from the destination to the source.
Mean_pkt_len	The average packet size (in bytes) within the flow.
Std_pkt_len	The variability (dispersion) of packet sizes in the flow.
Min_pkt_len	The smallest packet length observed in the flow.
Max_pkt_len	The largest packet length observed in the flow.
Sload	The data rate (bytes per second) of packets sent from the source host.
Dload	The data rate (bytes per second) of packets received by the destination host.

TH^2 term of recurrent networks, making the overall computational cost and time complexity of Transformer significantly higher than that of LSTM, GRU, and ARN. In summary, although ARN has slightly higher per-step computation than LSTM and GRU, its overall time complexity is comparable to theirs, whereas the computational overhead of Transformer is significantly higher than the aforementioned networks.

VII. SCHEME IMPLEMENTATION

In this section, we first explicitly describe the experimental environment, datasets and evaluation criteria in detail. Subsequently, we provide the experimental results and analysis.

A. Experimental Environment, Datasets, and Criteria

1) *Experimental Environment*: In this paper, all of the simulation experiments are carried out on a desktop equipped with a CUDA12.1 driver, cuDNN8.8.0.1 driver and Windows 10 operating system. The desktop is equipped with 32 GB main memory, an NVIDIA GeForce RTX 4070 graphics card and a 13th Gen Intel(R) Core(TM) i5-13600KF CPU that runs at 3.50 GHz. In addition, the experimental results of our proposed solution and comparative experiment are obtained by running in our experimental environment, the framework used in this experiment is based on pytorch2.1.0. To adapt to different datasets, distinct optimization strategies are applied: for the UNSW-NB15 and ICS-ADD datasets, we use the Adam optimizer with learning rates of 0.001 and 0.0001, respectively, whereas for the SWaT dataset, we use the SGD optimizer with a learning rate of 0.012. The ARN model is configured with a single-layer network structure and trained for 200 epochs, and the best-performing result during training is selected as the final experimental outcome.

2) *Experimental Datasets*: In the simulation experiments, we evaluate the proposed ARN method on ICS datasets (ICS-ADD and SWaT) to assess its applicability in industrial

control environments, and on the conventional network dataset UNSW-NB15 to further verify the generalizability of ARN.

ICS-ADD is a public anomaly detection dataset for industrial control system security research [42]. It records the system's behavior under various simulated attacks (e.g., The raw network traffic under DoS, MITM, malware intrusion, etc.) After preprocessing, we obtained a total of 23,023 samples, of which 9,264 were attack samples.

SWaT is a public security dataset for industrial water treatment platforms, built by iTrust, Centre for Research in Cyber Security, Singapore University of Technology and Design. Reference [43]. We chose the officially labeled "SWaT Dec 2015" for the experiments. SWaT dataset consists of two labels: "normal" and "attack", with a total of 449,919 pieces of data, of which 54,621 pieces are attack data.

UNSW-NB15 is a public conventional network intrusion detection dataset [45], [44], which contains a total of 2,540,044 pieces of data, including 300,000 pieces of attack data, with 9 type attacks, i.e., analysis, backdoors, DoS, exploits, fuzzers, generic, reconnaissance and shellcode.

Table IV presents the features used in the SWaT and UNSW-NB15 datasets (52 features for SWaT and 43 features for UNSW-NB15), while Table V lists the features extracted from the ICS-ADD dataset along with their descriptions (15 features for ICS-ADD). Unlike SWaT and UNSW-NB15, which provide official flow-level CSV files with well-documented feature definitions [43], [44], the ICS-ADD dataset is provided only in raw PCAP format [42]. Following the official instructions, we converted these PCAP files into flow-level CSV files and provided detailed descriptions for each extracted feature. It is worth noting that in our experiments, the UNSW-NB15 dataset had the IP features as well as the start time (Stime) and last time (Ltime) features removed. This is because these features could lead the model to memorize that certain attacks occur between specific IPs or during particular time periods, resulting in data leakage and negatively affecting the models generalization ability.

In the simulation experiment, we use an 8:2 ratio for the ICS-ADD dataset to determine the training and test sets. This is because the overall data size of ICS-ADD is relatively small, and sufficient test data is required to validate the effectiveness of our solution. For the SWaT dataset, we divide the selected SWaT data into a training set and a test set according to 8:2. This is because the sample distribution of the SWaT data set is uneven, and we need to include the corresponding attack data in the test set, so the selected test set has a large amount of data. Meanwhile, due to the large amount of UNSW-NB15 data, to facilitate the experiment,

TABLE VI

COMPARISON OF THE EFFECTIVENESS OF DIFFERENT METHODS ON THE ICS-ADD, SWAT AND UNSW-NB15 DATASETS. ARN APPLIES A ONE-TAILED WILCOXON SIGNED-RANK TEST TO EACH COMPARATIVE METHOD IN THE TABLE. ALL OBTAINED P-VALUES ARE $p = 0.03125 < 0.05$, INDICATING THAT THE PERFORMANCE IMPROVEMENT IS STATISTICALLY SIGNIFICANT; MOREOVER, SINCE THE p -VALUE IS COMPUTED BASED ON FIVE PAIRED VALIDATION RESULTS, $p = 0.03125$ IS THE SMALLEST P-VALUE ATTAINABLE BY THE TEST AT THIS SAMPLE SIZE. THE BEST RESULTS ARE EMPHASIZED IN BOLD

Method	ICS-ADD				SWaT				UNSW-NB15			
	Accuracy	Precision	Recall	F1_score	Accuracy	Precision	Recall	F1_score	Accuracy	Precision	Recall	F1_score
SDBSCAN [25]	78.37 ± 0.031	85.86 ± 0.009	78.37 ± 0.032	78.32 ± 0.032	94.79 ± 0.005	95.01 ± 0.121	94.79 ± 0.005	92.27 ± 0.009	93.33 ± 0.033	93.46 ± 0.024	93.33 ± 0.033	93.03 ± 0.038
DHMM [31]	78.34 ± 0.040	85.91 ± 0.015	78.34 ± 0.040	78.28 ± 0.040	93.41 ± 1.283	92.05 ± 0.964	93.41 ± 1.283	92.08 ± 0.276	96.94 ± 0.317	97.16 ± 0.438	96.94 ± 0.317	96.99 ± 0.335
GRU [37]	94.31 ± 0.184	94.66 ± 0.197	94.31 ± 0.184	94.23 ± 0.193	94.40 ± 0.373	92.73 ± 0.522	94.40 ± 0.373	93.02 ± 0.338	96.59 ± 0.772	95.38 ± 1.173	96.59 ± 0.772	95.74 ± 0.876
BiDLSTM [46]	94.57 ± 1.765	95.09 ± 1.521	94.57 ± 1.765	94.61 ± 1.731	94.98 ± 0.151	94.57 ± 0.385	94.98 ± 0.151	92.81 ± 0.431	96.86 ± 0.325	96.38 ± 0.301	96.86 ± 0.325	95.70 ± 0.547
ICS-CAD [36]	96.74 ± 1.768	96.87 ± 0.175	96.74 ± 1.768	96.73 ± 0.155	94.97 ± 0.013	94.28 ± 0.019	94.93 ± 0.114	92.82 ± 0.034	97.00 ± 0.171	96.25 ± 0.210	97.00 ± 0.171	96.42 ± 0.247
AE-LSTM [38]	95.05 ± 0.497	95.26 ± 0.478	95.05 ± 0.497	94.99 ± 0.504	95.05 ± 0.497	95.27 ± 0.477	95.05 ± 0.497	92.73 ± 0.111	97.14 ± 0.299	96.11 ± 0.648	97.14 ± 0.299	96.42 ± 0.585
DR-AD [35]	93.70 ± 2.817	94.35 ± 2.203	93.70 ± 2.817	93.55 ± 2.988	94.37 ± 0.351	90.93 ± 0.485	94.37 ± 0.351	92.22 ± 0.161	96.34 ± 0.261	93.46 ± 0.768	96.34 ± 0.261	94.86 ± 0.502
TNN [40]	98.62 ± 0.472	98.65 ± 0.443	98.62 ± 0.472	98.61 ± 0.473	95.30 ± 0.084	94.72 ± 0.301	95.30 ± 0.084	93.66 ± 0.206	97.13 ± 0.169	96.18 ± 0.384	97.13 ± 0.169	96.29 ± 0.227
ours	98.93 ± 0.329	98.95 ± 0.318	98.93 ± 0.329	98.93 ± 0.327	95.57 ± 0.080	95.17 ± 0.125	95.57 ± 0.080	94.17 ± 0.161	98.27 ± 0.046	98.01 ± 0.066	98.27 ± 0.046	98.02 ± 0.073

we randomly and sequentially selected 25% of the UNSW-NB15 dataset as experimental data, with a ratio of 10:1 for the training set and test set. In addition, to ensure the reliability of the experimental results, a cross-validation strategy is adopted during the experiments. Specifically, in each round, 10% of normal samples and 10% of abnormal samples are randomly selected in chronological order from the training set to construct the validation set, while the remaining data are used for training. For each round, the model is independently trained on a different training subset, with the corresponding validation set used to assist the training. The optimal model from each round is then evaluated on the test set, and the final result is reported as the average performance and standard deviation of the five models. This validation strategy is adopted because attack samples in industrial control datasets are often concentrated within specific time intervals. Using conventional cross-validation methods such as K-fold cross-validation may disrupt the temporal order if the data are shuffled before splitting, or result in training sets containing only a single class if the data are split sequentially. Both situations hinder the proper validation of the models effectiveness. The ICS-ADD, SWaT, and UNSW-NB15 datasets are all publicly available and can be obtained through official sources.

Finally, the final dataset splits are as follows: the UNSW-NB15 dataset contains 449,996 training samples, 50,004 validation samples, and 50,000 test samples, the SWaT dataset contains 323,942 training samples, 35,994 validation samples, and 89,983 test samples, and the ICS-ADD dataset contains 16,644 training samples, 1,850 validation samples, and 4,529 test samples. It should be noted that in the above datasets, flow records in ICS-ADD, SWaT, and UNSW-NB15 are arranged in chronological order by timestamps. We construct sequence samples of length $T = 8$, i.e., we extract 8 consecutive flow records from the time-ordered data to form an input sequence, which is used as the temporal modeling input to ARN.

3) *Experimental Datasets*: In the simulation experiments, we use *Accuracy*, *Precision*, *Recall*, and *F1_score* as experimental evaluation criteria to confirm the effectiveness of our proposed ARN-based intrusion detection method. We also use memory usage, detection time, and Floating Point Operations per Second (FLOPs) as evaluation metrics to test the efficiency of ARN.

B. Experimental Results and Analysis

In this part, we develop a prototype implementation to verify the effectiveness of our proposed method. Specifically, we

compare our ARN-based intrusion detection method with the SDBSCAN-based intrusion detection method [25], DHMM-based intrusion detection method [31], GRU-based intrusion detection method [37], AE-LSTM-based intrusion detection method [38], DR-AD-based intrusion detection method [35], BiDLSTM-based intrusion detection method [46], ICS-CAD-based intrusion detection method [36] and TNN-based intrusion detection method [40].

1) *Quantitative Performance*: We conduct the effectiveness verification of the proposed detection method based on the ICS datasets (ICS-ADD and SWaT) and conventional network datasets (UNSW-NB15), according to the experimental criteria defined in Section VII-A.3. The experimental results are shown in Table VI. It can be seen that the proposed ARN-based intrusion detection method achieves strong performance on both ICS datasets (ICS-ADD and SWaT) and the conventional network dataset (UNSW-NB15), significantly outperforming other comparative methods overall.

a) *On the ICS dataset ICS-ADD*: Traditional statistics-analysis methods such as SDBSCAN and time-series methods such as DHMM perform relatively poorly on ICS-ADD mainly because many attack stages in ICS-ADD are short, sparse, and stealthy under flow-level representations. Specifically, several attacks (e.g., Modbus scanning) primarily involve protocol-level manipulations and do not necessarily induce pronounced deviations in flow-level statistics, making them overlap with highly periodic benign Modbus polling traffic. As a result, SDBSCAN may absorb attack samples into the dominant benign high-density region, and DHMM becomes less discriminative due to weakened separability of the observation sequence, thus leading to more false negatives and consequently lower detection performance.

In deep learning solutions, the ARN-based intrusion detection method proposed in this paper achieves good performance across all evaluation metrics. This is because GRU-based intrusion detection method suffer from gate unit conflicts, making it difficult to effectively capture previous hidden state and consequently leading to decreased detection performance. The AE-LSTM-based intrusion detection method first extracts features through an autoencoder and then performs detection using an LSTM, its accuracy is affected not only by the quality of feature extraction in the AE but also by the gate conflict and feature forgetting issues inherent in the LSTM. The DR-AD-based intrusion detection method also requires feature extraction before detection, and its results are influenced by the quality of this feature extraction. Moreover, the DR-AD

model structure cannot exploit the temporal characteristics of network traffic for feature representation and learning, resulting in relatively lower overall detection metrics. Although the BiDLSTM-based intrusion detection method can model temporal features using a bidirectional structure, its performance is still affected by the inherent limitations of the LSTM architecture. Additionally, since attacks in industrial traffic typically follow a unidirectional temporal evolution, bidirectional modeling may dilute potential attack features, thereby weakening detection effectiveness. The main structure of the ICS-CAD-based intrusion detection method is still based on LSTM, and thus it is similarly constrained by gate mechanism issues, making it difficult to further improve detection accuracy. For the TNN-based intrusion detection method, although its core is composed of Transformers and can represent industrial control traffic features from a global dependency perspective through the attention mechanism, in ICN, most anomalies occur only within a limited time window, manifesting as local changes in packet frequency, latency, or control commands. Moreover, attack behaviors often exhibit unidirectional temporal evolution, so the global attention mechanism of the Transformer may introduce redundant dependencies, reducing the models sensitivity to local anomalous features and consequently affecting detection accuracy. In contrast, the ARN-based intrusion detection method effectively alleviates conflicts among gate units and can more reasonably manage the relationship between previous hidden state and the retention of current input, thus more accurately representing the unidirectional temporal evolution in traffic. Therefore, compared with other existing methods, the proposed method achieves the best intrusion detection performance.

We also observe that the DR-AD-based intrusion detection method and BiDLSTM-based intrusion detection methods exhibit relatively high standard deviations on the ICS-ADD dataset. We speculate that this is mainly related to the fact that attack behaviors in ICS-ADD are concentrated within specific time periods. In other words, attack samples occur continuously only within certain time windows, and random cross-validation may allocate part of these periods to the validation set, resulting in incomplete attack sequences during training. Such missing samples can be partially compensated through temporal modeling mechanisms. However, the DR-AD approach fails to effectively capture temporal dependencies, and since attacks in industrial datasets typically follow a unidirectional temporal evolution, the bidirectional modeling of BiLSTM may dilute the attack features. Moreover, the high expressive capacity of deep learning methods makes them sensitive to the division of the training set, leading to larger performance fluctuations under different splits and, consequently, relatively higher standard deviations.

b) On the ICS dataset SWaT: The SWaT dataset is highly imbalanced, which increases the difficulty of distinguishing between certain normal and abnormal traffic, resulting in overall performance of various detection methods being relatively close. Nevertheless, according to the evaluation metrics described in Section VII-A3, our ARN-based intrusion detection method still achieves the best performance across all metrics. We can also be observed that the SDBSCAN-based

intrusion detection method outperforms some deep learning methods. This is mainly because SDBSCAN relies on the distribution of samples in the feature space, and the high-dimensional feature space of SWaT allows attack samples to form distinguishable clusters. Even when the number of attack samples is small, statistics-analysis methods are less affected. In contrast, deep learning methods trained on imbalanced samples are more susceptible to distribution shifts caused by a small number of attack samples, leading to unstable model fitting. Therefore, on SWaT, statistics-analysis methods achieve better performance compared to some deep learning methods. The performance analysis of other methods aligns closely with the observations made on the ICS-ADD dataset.

c) On the conventional network dataset UNSW-NB15:

The UNSW-NB15 is a public dataset focused on conventional network traffic. Experimental results on this dataset show that even when dealing with regular network traffic, our proposed ARN-based intrusion detection method outperforms other comparative methods. It can also be observed that, in this case, the time-series intrusion detection method based on DHMM achieves better performance than some deep learning approaches. This is mainly because the UNSW-NB15 dataset consists of conventional network traffic, where temporal dependencies primarily manifest as continuous communication records within individual sessions, exhibiting locally continuous time series characteristics. Such local continuity aligns well with the Markov assumption, enabling DHMM to more effectively capture state transition patterns and achieve superior detection performance. In contrast, the DR-AD-based intrusion detection method fails to effectively learn temporal dependencies, the GRU-based intrusion detection method may be affected by gate conflicts when capturing temporal relationships, and the BiDLSTM-based intrusion detection method is constrained by the gate conflicts inherent in LSTM, and its bidirectional modeling mechanism may dilute abnormal attack features when learning network data with unidirectional temporal characteristics, resulting in a slight performance degradation. Moreover, the local continuity of the UNSW-NB15 dataset also limits the learning effectiveness of Transformers. Since Transformers emphasize global dependencies, modeling locally continuous sequences can introduce redundant features, thereby reducing the accuracy of TNN-based intrusion detection method. The performance analysis of other methods is largely consistent with our observations on the ICS-ADD dataset.

Overall, the results show that the ARN-based intrusion detection method achieves strong performance on both industrial control network traffic and conventional network traffic. In particular, the high detection accuracy on the ICS-ADD and UNSW-NB15 datasets mainly stems from the synergy between the dataset characteristics and the design of ARN. The ICS-ADD dataset contains highly periodic industrial traffic, while the UNSW-NB15 dataset still preserves locally continuous session-level temporal structures under the adopted flow-level representation. With the aid of the S-ATT mechanism, ARN can explicitly model the relationship between the previous hidden state and the current input, thereby more effectively extracting temporal features from the traffic and identifying

TABLE VII

ABLATION EXPERIMENT OF THE S-ATT MODULE IN ARN, WHERE W/O(S-ATT) REPRESENTS THE MODEL WITHOUT THE S-ATT MODULE. THE BEST RESULTS ARE EMPHASIZED IN BOLD

Method	ICS-ADD			
	Accuracy	Precision	Recall	F1_score
w/o(S-ATT)	93.58 $\pm$ 2.725	94.27 $\pm$ 2.276	93.58 $\pm$ 2.725	93.43 $\pm$ 2.824
ours	98.93 $\pm$ 0.329	98.95 $\pm$ 0.318	98.93 $\pm$ 0.329	98.93 $\pm$ 0.327
Method	SWaT			
	Accuracy	Precision	Recall	F1_score
w/o(S-ATT)	94.56 $\pm$ 1.141	93.70 $\pm$ 1.530	94.56 $\pm$ 1.141	93.08 $\pm$ 0.647
ours	95.57 $\pm$ 0.080	95.17 $\pm$ 0.125	95.57 $\pm$ 0.080	94.17 $\pm$ 0.161
Method	UNSW-NB15			
	Accuracy	Precision	Recall	F1_score
w/o(S-ATT)	90.70 $\pm$ 0.046	89.40 $\pm$ 0.196	90.70 $\pm$ 0.046	89.13 $\pm$ 0.074
ours	98.27 $\pm$ 0.046	98.01 $\pm$ 0.066	98.27 $\pm$ 0.046	98.02 $\pm$ 0.073

TABLE VIII

ABLATION STUDY BY REMOVING CERTAIN METADATA FEATURES, WHERE W/O (-) DENOTES REMOVING THE METADATA FEATURES SPECIFIED IN THE PARENTHESES. THE BEST RESULTS ARE EMPHASIZED IN BOLD

Method	ICS-ADD			
	Accuracy	Precision	Recall	F1_score
w/o(Sport, Dsport)	98.62 $\pm$ 0.429	98.64 $\pm$ 0.411	98.62 $\pm$ 0.429	98.62 $\pm$ 0.427
ours	98.93 $\pm$ 0.329	98.95 $\pm$ 0.318	98.93 $\pm$ 0.329	98.93 $\pm$ 0.327
Method	UNSW-NB15			
	Accuracy	Precision	Recall	F1_score
w/o(Sport, Dsport, Service)	98.03 $\pm$ 0.027	97.69 $\pm$ 0.062	98.03 $\pm$ 0.027	97.78 $\pm$ 0.032
ours	98.27 $\pm$ 0.046	98.01 $\pm$ 0.066	98.27 $\pm$ 0.046	98.02 $\pm$ 0.073

TABLE IX

HYPERPARAMETER ANALYSIS WAS PERFORMED ON THE SEQUENCE LENGTH T. THE BEST RESULTS ARE EMPHASIZED IN BOLD

The length of T	ICS-ADD			
	Accuracy	Precision	Recall	F1_score
T=4	95.27 $\pm$ 0.215	95.51 $\pm$ 0.197	95.27 $\pm$ 0.215	95.23 $\pm$ 0.226
T=8 (ours)	98.93 $\pm$ 0.329	98.95 $\pm$ 0.318	98.93 $\pm$ 0.329	98.93 $\pm$ 0.327
T=12	98.60 $\pm$ 0.295	98.63 $\pm$ 0.280	98.60 $\pm$ 0.295	98.60 $\pm$ 0.297

anomalous deviations, while also avoiding the gating conflict problem commonly found in traditional recurrent models. As a result, high detection accuracy is achieved on these two datasets. In contrast, although the SWaT dataset shares certain similarities with ICS-ADD in terms of industrial traffic characteristics, its severe class imbalance increases the difficulty of distinguishing some normal flows from anomalous flows, thereby leading to relatively lower detection accuracy on this dataset.

2) *Ablation Study*: In this section, we conduct an ablation study on S-ATT, the core module of the ARN method, and the port/service metadata features that may lead to data contamination. We also perform a hyperparameter analysis on the sequence length T.

a) *w/o(S-ATT)*: The experimental results are shown in Table VII. It can be seen that when the S-ATT module is removed, and the previous hidden state is directly added to the current input for representation learning, the model performance on all three datasets decreases significantly. This is because the model without the S-ATT module cannot effectively analyze the correlation between the previous hidden state and the current input, making it difficult to determine which previous hidden state should be retained or discarded. As a result, a large amount of redundant previous hidden state is introduced into the current representation, interfering with the learning of temporal relationships and leading to a noticeable decrease in detection accuracy. Moreover, since the model lacks the ability to adaptively adjust the retention degree of previous hidden state, it is sensitive to variations in temporal

segmentation of samples, resulting in significant performance fluctuations during cross-validation, particularly exhibiting large standard deviations on the ICS dataset ICS-ADD. In contrast, the ARN with the S-ATT module can effectively analyze the relationship between the previous hidden state and the current input. By constructing a supplementary matrix to enhance the current input, it realizes a dynamic reset of the hidden state. The reset current hidden state contains the important previous hidden state that is unknown in the current input and highlights the features in the current input that are related to previous hidden state, thereby significantly improving the overall performance of the model. In addition, based on five paired experimental results, we perform one-tailed Wilcoxon signed-rank tests between ARN(ours) and w/o(S-ATT), and all p-values are 0.03125(< 0.05), indicating statistical significance.

b) *w/o(port/service metadata features)*: Since our experiments adopt a flow-level feature representation, the constructed feature set often may contain port/service-related metadata features (e.g., Sport/Dport and the Service proxy). These fields may become strongly correlated with labels under specific data splits, thereby significantly affecting the classifiers decision and introducing the risk of metadata contamination [48]. To verify that the performance gain of ARN does not mainly rely on such port/service fields, we conduct a feature-level ablation study by removing these metadata features: specifically, we remove Sport and Dport from the ICS dataset ICS-ADD, and remove Sport, Dport, and Service from the conventional network dataset UNSW-NB15. Since the SWaT dataset does not contain these port-/service-related features, no corresponding removal experiment is performed on SWaT. As shown in Table VIII, the experimental results show that the performance change after removing the above features remains within 0.5%, indicating that ARNs effectiveness primarily stems from its ability to model temporal behavioral patterns in traffic rather than exploiting port-/service-related metadata features. In addition, based on five paired experimental results, we perform one-tailed Wilcoxon signed-rank tests between ARN using the full feature set (ours) and w/o(port/service metadata features), and all p-values are 0.03125(< 0.05), indicating statistical significance.

c) *Hyperparameter analysis*: Given that the ICS-ADD dataset is relatively new and has a fairly balanced sample distribution, we perform hyperparameter analysis on the sequence length T using the ICS-ADD dataset, and the results are shown in Table IX. It can be observed that the model performance exhibits a rise-then-fall trend as T increases: when T= 4, the sequence is too short to provide sufficient historical context, making it difficult to adequately model temporal dependencies and resulting in inferior performance; when T= 8, the model benefits from richer temporal information and achieves the best performance; when T is further increased (e.g., T= 12), overly long sequences tend to introduce redundant or irrelevant historical information and may even span transitional states, thereby weakening the discriminability of local anomalous patterns and leading to performance degradation. These results indicate that a moderate sequence length achieves a better balance between sufficient temporal context and noise

TABLE X

COMPUTATIONAL COSTS OF DIFFERENT METHODS. THE TIME COST IS MEASURED IN SECONDS (S), THE MEMORY USAGE IS MEASURED IN MEGABYTES (MB), AND THE FLOPS ARE MEASURED IN MILLIONS (M). TO ENSURE A FAIR COMPARISON, WE EVALUATE THE COMPUTATIONAL COST OF EACH METHOD UNDER THE SAME IDENTICAL PARAMETER SETTINGS. THE BEST RESULTS ARE EMPHASIZED IN BOLD

Method	ICS-ADD			SWaT			UNSW-NB15		
	Memory	Time	FLOPs	Memory	Time	FLOPs	Memory	Time	FLOPs
SDBSCAN [25]	5.148 $\pm$ 0.297 MB	0.503 $\pm$ 0.012 s	-	84.368 $\pm$ 1.367 MB	10.536 $\pm$ 0.056 s	-	41.476 $\pm$ 0.729 MB	5.508 $\pm$ 0.087 s	-
DHMM [31]	0.116 $\pm$ 0.009 MB	0.081 $\pm$ 0.001s	-	30.114 $\pm$ 0.272 MB	2.766 $\pm$ 0.015s	-	12.954 $\pm$ 0.105 MB	1.610 $\pm$ 0.002s	-
GRU [37]	1711.926 $\pm$ 0.233 MB	0.907 $\pm$ 0.011 s	2.139 M	1766.000 $\pm$ 0.473 MB	3.605 $\pm$ 0.035 s	2.494 M	1732.856 $\pm$ 0.226 MB	2.463 $\pm$ 0.013 s	2.411 M
BiDLSTM [46]	1706.920 $\pm$ 0.343 MB	0.986 $\pm$ 0.008 s	5.846 M	1762.012 $\pm$ 0.703 MB	3.748 $\pm$ 0.045 s	6.793 M	1727.900 $\pm$ 0.449 MB	2.535 $\pm$ 0.014 s	6.566 M
ICS-CAD [36]	1710.346 $\pm$ 0.789 MB	0.911 $\pm$ 0.007 s	2.856 M	1765.738 $\pm$ 0.706 MB	3.633 $\pm$ 0.020 s	3.329 M	1732.534 $\pm$ 0.321 MB	2.484 $\pm$ 0.011 s	3.217 M
AE-LSTM [38]	1715.378 $\pm$ 0.481 MB	0.949 $\pm$ 0.007 s	2.766 M	1852.890 $\pm$ 0.907 MB	3.961 $\pm$ 0.013 s	2.909 M	1788.496 $\pm$ 0.413 MB	2.681 $\pm$ 0.025 s	2.912 M
DR-AD [35]	1710.618 $\pm$ 0.477 MB	0.881 $\pm$ 0.028 s	0.006 M	1808.516 $\pm$ 1.179 MB	3.624 $\pm$ 0.049 s	0.013 M	1763.256 $\pm$ 0.548 MB	2.517 $\pm$ 0.020 s	0.016 M
TNN [40]	2997.188 $\pm$ 0.854 MB	1.705 $\pm$ 0.024 s	5.189 M	3051.210 $\pm$ 0.961 MB	4.395 $\pm$ 0.145 s	5.307 M	3019.196 $\pm$ 0.985 MB	3.302 $\pm$ 0.018 s	5.279 M
ours	1712.614 $\pm$ 0.391 MB	0.920 $\pm$ 0.013 s	3.912 M	1766.516 $\pm$ 0.409 MB	3.651 $\pm$ 0.014 s	4.030 M	1734.212 $\pm$ 0.398 MB	2.498 $\pm$ 0.016 s	4.004 M

suppression; therefore, we set $T = 8$ by default. Moreover, based on five paired experimental results, one-tailed Wilcoxon signed-rank tests are performed by taking ARN with $T = 8$ (ours) as the reference and comparing it with ARN with $T = 4$ and $T = 12$. All p-values are $0.03125 (< 0.05)$, indicating statistical significance.

3) *Analysis of Computational Overhead*: This section presents experiments evaluating the computational overhead of the proposed ARN method and the baseline algorithms in the experimental environment. The evaluation metrics include FLOPs, memory consumption, and runtime, with runtime covering the overhead of preprocessing and model inference for all test data. The results are shown in Table III. Since the main computational cost of SDBSCAN and DHMM does not come from large-scale floating-point matrix operations, FLOPs are not used as a measure of computational complexity. It can be seen that the SDBSCAN-based intrusion detection method and the DHMM-based intrusion detection method consume much less memory than all deep learning models. This is mainly because both are lightweight algorithms based on statistical and probabilistic inference, with very few parameters. In contrast, deep learning models need to cache a large number of weights and hidden states within the network. In particular, Transformer models must simultaneously process and cache the dependencies across all time steps, resulting in higher memory requirements. Therefore, the overall memory consumption of deep learning models is significantly higher than that of traditional statistics-analysis or time-series algorithms. Additionally, although SDBSCAN has relatively low memory overhead, its DBSCAN-based clustering process incurs a high computational cost when handling high-dimensional data [49], causing its runtime on the SWaT dataset to exceed 10 seconds. This is mainly because the efficiency of neighborhood search in high-dimensional space decreases, leading to a significant increase in the complexity of distance computation and density estimation. The DHMM-based intrusion detection method shows the best performance in terms of memory and runtime overhead, reflecting higher computational efficiency, its detection performance on the ICS-ADD and SWaT datasets remains low. Therefore, it is difficult to meet the practical detection requirements in industrial scenarios, making it unsuitable as a mainstream intrusion detection solution.

In deep learning-based intrusion detection methods, although memory overhead is typically much higher than that of traditional methods such as SDBSCAN and DHMM, the decreasing cost of memory allows small-scale deep learning

models to be deployed in industrial networks. According to the experimental results, although our proposed ARN-based intrusion detection method exhibits higher FLOPs than GRU, ICS-CAD, AE-LSTM, and DR-AD, its memory and runtime overhead are comparable to these methods, and its memory consumption is even lower than that of AE-LSTM and DR-AD on the SWaT and UNSW-NB15 datasets. This is mainly because AE-LSTM and DR-AD adopt a two-stage structure of feature extraction + detection, in which intermediate feature variables must be retained between the two modules, thereby increasing the overall memory usage. Moreover, since the feature extraction and detection models are independent, additional feature transmission and tensor loading operations are required during inference, which further increase the total runtime. This also explains why AE-LSTM exhibits higher time overhead than ARN, and why DR-AD, despite having fewer FLOPs, still shows a runtime similar to that of ARN.

The ICS-CAD intrusion detection method, based on LSTM, and the GRU-based intrusion detection method both have lower theoretical FLOPs than the proposed ARN method, but their actual runtime is close to ARN. From the perspective of theoretical FLOPs, ARN exhibits slightly higher per-step computation than GRU and LSTM, mainly due to additional linear transformations and attention operations in the S-ATT module on the hidden state and input vectors. However, in practice, although ARN still processes sequences step by step, the matrix multiplications, dot products, and weighted sums within each time step can be highly parallelized on SIMD or GPU architectures, fully leveraging hardware capabilities. In contrast, because the gating mechanism introduces more gate-related branches and element-wise operations, GRU/LSTM need to perform more matrix multiplications and nonlinear computations at each time step, which may increase the accumulated sequence-level latency. Therefore, despite slightly higher theoretical computation, ARNs actual runtime remains close to that of GRU and LSTM, which explains the observed experimental results. In addition, ARN incurs slightly higher memory usage than GRU and LSTM, primarily because the S-ATT module generates additional intermediate vectors and attention matrices at each time step to compute attention weights between the hidden state and input. In contrast, GRU and LSTM only need to store a small number of hidden states and gate-related matrices per step, resulting in lower memory consumption. We also observed that GRUs measured memory usage is slightly higher than LSTM, which may be attributed to implementation-dependent caching

and kernel fusion behaviors under our memory measurement setting.

Due to its bidirectional structure, BiDLSTM has the highest FLOPs among all compared methods. However, inference time does not necessarily scale strictly linearly with FLOPs. In our implementation, BiDLSTM performs gate computations step by step over time and involves multiple matrix multiplications and activation operations. Its runtime is also influenced by factors such as GPU parallel efficiency, operator scheduling overhead, and memory bandwidth. Therefore, although its inference time is relatively long, it does not increase proportionally with FLOPs. For memory consumption, we measure memory usage as the difference between the beginning and the end of the detection phase. Owing to the frameworks memory reuse/caching mechanism, the net memory increase may vary across models, making the measured memory overhead of BiDLSTM slightly lower than that of ICS-CAD.

In contrast, the TNN method employs a Transformer architecture. Its multi-head self-attention module computes pairwise attention among all elements in the sequence at each time step, resulting in $O(T^2D)$ computational complexity and generating a large number of intermediate attention matrices. In addition, the feed-forward sublayers and projection operations in multi-head attention introduce additional computation and intermediate activations, significantly increasing both FLOPs and memory consumption. In practice, these operations not only lengthen the overall inference time but also substantially raise the memory footprint, increasing deployment costs in industrial network environments. Given that industrial intrusion detection often requires real-time or near-real-time response, the high runtime and memory demands of Transformer may limit its practical applicability in such scenarios.

In summary, although traditional methods such as SDB-SCAN and DHMM have relatively low memory overhead, SDBSCAN exhibits a significant increase in runtime when processing high-dimensional data, and both types of methods show suboptimal detection performance on ICS datasets (ICS-ADD and SWaT), making them inadequate for the demands of modern industrial traffic detection. Among the compared deep learning methods, although the proposed ARN incurs slightly higher FLOPs on both the ICS datasets (ICS-ADD and SWaT) and the conventional network dataset (UNSW-NB15), its inference latency and memory overhead are comparable to those of GRU and ICS-CAD, and its computational efficiency during the inference phase is significantly higher than TNN. Moreover, ARN demonstrates noticeably superior detection performance compared to other methods, highlighting its high applicability and practical value as an industrial intrusion detection solution.

To further evaluate the feasibility of deploying the proposed method in practice, we analyze its latency overhead. According to Section VII-A, the number of test samples in the ICS datasets (ICS-ADD and SWaT) and the conventional network dataset (UNSW-NB15) are 4,529, 89,983, and 50,000, respectively. Based on the Time column in Table X, the per-flow-sample latency (includes data preprocessing and model inference time, and we use milliseconds to represent this

time) of the ARN model is approximately 0.203 ms for ICS-ADD, 0.041 ms for SWaT, and 0.05 ms for UNSW-NB15. Considering that communication latency in industrial networks is typically within 10 ms [47], and taking queuing effects into account under conservative assumptions (e.g., using only 80% of the theoretical throughput to accommodate additional I/O and caching overhead), the ARN model achieves a maximum processing capacity of approximately 3,941 and 19,512 samples per second on the ICS datasets ICS-ADD and SWaT, respectively. This indicates that ARN can satisfy the real-time requirements of industrial networks. Moreover, on the conventional network dataset UNSW-NB15, ARN achieves a maximum processing capacity of approximately 16,000 samples per second, indicating that it also supports efficient online processing in non-industrial network environments.

VIII. CONCLUSION AND FUTURE WORK

In the ICN, network traffic data exhibits characteristics of large scale, irregularity, multiple features, temporal correlation and high dimensionality, resulting in low accuracy of existing intrusion detection methods. In this paper, we focus on designing an accurate intrusion detection method for ICN. First, we design a novel RNN model, namely, ARN. Subsequently, we adopt ARN to build a novel intrusion detection method for ICN. By taking the advantage of ARN, our proposed method greatly improves the accuracy of intrusion detection. Finally, we implement our proposed ARN-based intrusion detection method and provide a performance evaluation. Compared to existing intrusion detection methods, our proposed method achieved accuracy rates of 98.93%, 95.57%, and 98.27% on the ICS datasets (ICS-ADD and SWaT) and the conventional network dataset (UNSW-NB15), respectively.

In practical application, ARN may have the same problem as other RNN methods, that is, in the process of characterizing and learning previous hidden state, ARN and existing RNN methods will eliminate some previous hidden state that is irrelevant to the current by calculating the association between previous hidden state and current input. However, such information may still be highly valuable for future inputs, leading to information loss in modeling temporal dependencies. Moreover, network security managers not only expect real-time intrusion detection but also seek to anticipate the network security situation at the next time step, enabling proactive defensive measures. In this paper, we focus solely on intrusion detection for ICN, without addressing network security situation forecasting. Hence, future work will explore these limitations in the ARN model and investigate predictive security situation assessment for ICN.

REFERENCES

- [1] B. Galloway and G. P. Hancke, "Introduction to industrial control networks," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 2, pp. 860–880, 2nd Quart., 2013.
- [2] T. Cao et al., "Reliable and efficient multimedia service optimization for edge computing-based 5G networks: Game theoretic approaches," *IEEE Trans. Netw. Service Manage.*, vol. 17, no. 3, pp. 1610–1625, Sep. 2020.
- [3] N. Koursiompas, S. Barmounakis, I. Stavarakakis, and N. Alonistioti, "AI-driven, context-aware profiling for 5G and beyond networks," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1036–1048, Jun. 2022.

- [4] D. Ding, Q.-L. Han, Z. Wang, and X. Ge, "A survey on model-based distributed control and filtering for industrial cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 15, no. 5, pp. 2483–2499, May 2019.
- [5] Z. Lv, Y. Han, A. K. Singh, G. Manogaran, and H. Lv, "Trustworthiness in industrial IoT systems based on artificial intelligence," *IEEE Trans. Ind. Informat.*, vol. 17, no. 2, pp. 1496–1504, Feb. 2021.
- [6] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024.
- [7] J. Chen, Y. Chen, S. Cai, S. Yin, L. Zhao, and Z. Zhang, "An optimized feature extraction algorithm for abnormal network traffic detection," *Future Gener. Comput. Syst.*, vol. 149, pp. 330–342, Dec. 2023.
- [8] X. Etxezarreta, I. Garitano, M. Iturbe, and U. Zurutuza, "Low delay network attributes randomization to proactively mitigate reconnaissance attacks in industrial control systems," *Wireless Netw.*, vol. 30, no. 6, pp. 5077–5091, Aug. 2024.
- [9] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng, "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search," *Comput. Secur.*, vol. 143, Aug. 2024, Art. no. 103910.
- [10] Q. Wu, Z. Jiang, K. Hong, H. Liu, L. T. Yang, and J. Ding, "Tensor-based recurrent neural network and multi-modal prediction with its applications in traffic network management," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 780–792, Mar. 2021.
- [11] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Sep. 2014.
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [13] M. Morchid, "Parsimonious memory unit for recurrent neural networks with application to natural language processing," *Neurocomputing*, vol. 314, pp. 48–64, Nov. 2018.
- [14] H. Fanta, Z. Shao, and L. Ma, "Forget the forget gate: Estimating anomalies in videos using self-contained long short-term memory networks," presented at the 7th Comput. Graph. Int. Conf., Oct. 2020.
- [15] J. Aitchison, "The statistical analysis of compositional data," *J. Roy. Stat. Society: Ser. B (Methodol.)*, vol. 44, no. 2, pp. 139–160, 1982.
- [16] Y. Zou et al., "Complex network approaches to nonlinear time series analysis," *Phys. Rep.*, vol. 787, pp. 1–97, Sep. 2019.
- [17] S. Dong, P. Wang, and K. Abbas, "A survey on deep learning and its applications," *Comput. Sci. Rev.*, vol. 40, Jul. 2021, Art. no. 100379.
- [18] W. Wu et al., "Sliding window optimized information entropy analysis method for intrusion detection on in-vehicle networks," *IEEE Access*, vol. 6, pp. 45233–45245, 2018.
- [19] Y. Hu et al., "Detecting stealthy attacks on industrial control systems using a permutation entropy-based method," *Future Gener. Comput. Syst.*, vol. 108, pp. 1230–1240, Sep. 2020.
- [20] M. Zhou, Z. Zhang, and L. Xie, "Permutation entropy based detection scheme of replay attacks in industrial cyber-physical systems," *J. Franklin Inst.*, vol. 358, no. 7, pp. 4058–4076, May 2021.
- [21] D. Liao, R. Zhou, H. Li, M. Zhang, and X. Chen, "GE-IDS: An intrusion detection system based on grayscale and entropy," *Peer-Peer Netw. Appl.*, vol. 15, no. 3, pp. 1–14, May 2022.
- [22] M. Ahmed, A. Naser Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, Jan. 2016.
- [23] W. Liang, K.-C. Li, J. Long, X. Kui, and A. Y. Zomaya, "An industrial network intrusion detection algorithm based on multifeature data clustering optimization model," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2063–2071, Mar. 2020.
- [24] H. Mittal, A. K. Tripathi, A. C. Pandey, M. D. Alshehri, M. Saraswat, and R. Pal, "A new intrusion detection method for cyber-physical system in emerging industrial IoT," *Comput. Commun.*, vol. 190, pp. 24–35, Jun. 2022.
- [25] Z. D. Zhang, H. T. Shen, and S. J. Wei, "Network anomaly detection based on traffic clustering with group-entropy similarity," presented at the 9th Int. Symp. Netw., Comput. Commun., Shenzhen, China, 2022.
- [26] M. Bozdal, M. Samie, and I. K. Jennions, "WINDS: A wavelet-based intrusion detection system for controller area network (CAN)," *IEEE Access*, vol. 9, pp. 58621–58633, 2021.
- [27] K. Miao, X. Shi, and W.-A. Zhang, "Attack signal estimation for intrusion detection in industrial control system," *Comput. Secur.*, vol. 96, Sep. 2020, Art. no. 101926.
- [28] R. F. Fouladi, C. E. Kayatas, and E. Anarim, "Frequency based DDoS attack detection approach using naive Bayes classification," presented at the 39th Int. Conf. Telecommun. Signal Process. (TSP), Jun. 2016.
- [29] X. Ye, J. Lan, and W. Huang, "Network traffic anomaly detection based on self-similarity using FRFT," presented at the 4th Int. Conf. Softw. Eng. Service Sci., Beijing, China, May 2013.
- [30] D. Huang, X. Shi, and W.-A. Zhang, "False data injection attack detection for industrial control systems based on both time- and frequency-domain analysis of sensor data," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 585–595, Jan. 2021.
- [31] B. Zhang, X. Liu, H. Zheng, and Y. Song, "Hidden Markov model-based cyberattack prediction in power systems," *IEEE Trans. Smart Grid*, vol. 16, no. 2, pp. 1694–1705, Mar. 2025.
- [32] Y. Li et al., "Robust detection for network intrusion of industrial IoT based on multi-CNN fusion," *Measurement*, vol. 154, Jun. 2020, Art. no. 107450.
- [33] M. Abdelaty, R. Doriguzzi-Corin, and D. Siracusa, "DAICS: A deep learning solution for anomaly detection in industrial control systems," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 1117–1129, Apr. 2022.
- [34] A. Yang, Y. Zhuansun, C. Liu, J. Li, and C. Zhang, "Design of intrusion detection system for Internet of Things based on improved BP neural network," *IEEE Access*, vol. 7, pp. 106043–106052, 2019.
- [35] K. Sood, M. R. Nosouhi, D. D. N. Nguyen, F. Jiang, M. Chowdhury, and R. Doss, "Intrusion detection scheme with dimensionality reduction in next generation networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 965–979, 2023.
- [36] Z. Jadidi, J. Hagemann, and D. Quevedo, "Multi-step attack detection in industrial control systems using causal analysis," *Comput. Ind.*, vol. 142, Sep. 2022, Art. no. 103741.
- [37] C. Xu, J. Shen, X. Du, and F. Zhang, "An intrusion detection system using a deep neural network with gated recurrent units," *IEEE Access*, vol. 6, pp. 48697–48707, 2018.
- [38] E. Mushtaq et al., "A two-stage intrusion detection system with auto-encoder and LSTMs," *Appl. Soft Comput.*, vol. 121, Sep. 2022, Art. no. 108768.
- [39] D. Zhang, M. Wang, Y. Bu, J. Yu, and L. Yang, "PdGAT-ID: An intrusion detection method for industrial control systems based on periodic extraction and spatiotemporal graph attention," *Comput. Secur.*, vol. 149, Feb. 2025, Art. no. 104210.
- [40] S. Ullah et al., "TNN-IDS: Transformer neural network-based intrusion detection system for MQTT-enabled IoT networks," *Comput. Netw.*, vol. 237, Dec. 2023, Art. no. 110072.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inform. Process. Syst. (NIPS)*, 2017.
- [42] G. B. Gaggero, A. Armellini, G. Portomauro, and M. Marchese, "Industrial control system-anomaly detection dataset (ICS-ADD) for cyber-physical security monitoring in smart industry environments," *IEEE Access*, vol. 12, pp. 64140–64149, 2024.
- [43] J. Goh et al., "A dataset to support research in the design of secure water treatment systems," presented at the Crit. Inf. Infrastructures Secur., Paris, France, 2016.
- [44] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," presented at the Mil. Commun. Inf. Syst. Conf., Canberra, ACT, Australia, 2015.
- [45] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. Journal: A Global Perspective*, vol. 25, nos. 1–3, pp. 18–31, Apr. 2016.
- [46] Y. Imrana, Y. Xiang, L. Ali, and Z. Abdul-Rauf, "A bidirectional LSTM deep learning approach for intrusion detection," *Expert Syst. Appl.*, vol. 185, Dec. 2021, Art. no. 115524.
- [47] K. Das and P. Havinga, "Evaluation of DECT for low latency real-time industrial control networks," presented at the IEEE Int. Conf. Sens., New Orleans, LA, USA, Jun. 2013.
- [48] L. D'hooge, M. Verkerken, B. Volckaert, T. Wauters, and F. D. Turck, "Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*. Cham, Switzerland: Springer, 2022.
- [49] Y. Lv et al., "An efficient and scalable density-based clustering algorithm for datasets with complex structures," *Neurocomputing*, vol. 171, pp. 9–22, Jan. 2016.